

Time-Aware and Context-Sensitive Ensemble Learning for Sequential Data

Arda Fazla¹, Mustafa E. Aydin², and Suleyman S. Kozat³, *Senior Member, IEEE*

Abstract—We investigate sequential time series data through ensemble learning. Conventional ensemble algorithms and the recently introduced ones have provided significant performance improvements in widely publicized time series prediction competitions for stationary data. However, recent studies are inadequate in capturing the temporally varying statistics for nonstationary data. To this end, we introduce a novel approach using a metalearner that effectively combines base learners in both a time-varying and context-dependent manner. Our approach is based on solving a weight optimization problem that minimizes a specific loss function with constraints on the linear combination of the base learners. The constraints are theoretically analyzed under known statistics and integrated into the learning procedure of the metalearner as part of the optimization in an automated manner. We demonstrate significant performance improvements on real-life data and well-known competition datasets over the widely used conventional ensemble methods and the state-of-the-art forecasting methods in the machine learning literature. Furthermore, we openly share the source code of our method to facilitate further research and comparison.

Impact Statement—The sequential prediction task has found immense application areas, such as weather, electricity, and medical treatments. Due to the “no free lunch” theorem, there is no single best model to address each problem well. However, superior performance gains are obtainable when many models come together efficiently, i.e., the well-known ensembling approach. Our innovation to this end involves a generic context-sensitive and time-dependent learning framework that addresses the shortcomings of the current approaches suffering from biasing toward one of the base models and/or ignoring the temporally changing dynamics of the target sequence. Our work provides an efficient way for combining base models for prediction and a generic framework for solving practical time series problems.

Index Terms—Artificial neural network (ANN), ensemble learning, light gradient boosting machine (LightGBM), prediction/regression, time series.

I. INTRODUCTION

A. Preliminaries

WE STUDY nonlinear time series regression (prediction, learning, and similar) where we receive a target sequence and predict the next samples of the given sequence based

on the knowledge of the history, which includes the past values of the sequence and a data-specific side-information vector. This problem is extensively studied in the machine learning, computational learning theory, and signal processing literature under different applications [1], [2], [3]. There are essentially two common approaches to forecasting time series, which are generating forecasts from a single machine-learning model and combining forecasts of multiple models. The latter approach referred to as “ensemble learning,” is often superior to its individual counterparts, provided that the individual learners are accurate enough and diverse [4], [5]. The predominant performance of ensemble models has been showcased in many recent well-known forecasting competitions [6], [7].

Common strategies for ensembling include boosting [8] and bagging [9]. The boosting method combines a set of weak models in a sequential manner, where each model tries to compensate for the errors of the previous models. In bagging, also referred to as bootstrap aggregation, the weak models are trained independently—with samples and/or features reduced—and then combined by a deterministic process. In this article, we are interested in combining a set of machine learning models using a weighted average of their predictions, often referred to as stacking [10]. Combining forecasts using a weighted average so that a certain performance metric is minimized has been shown to be a successful approach for hindering the effects of selecting a weak base model for the specified learning task [11], and has been extensively studied in the literature [10], [11], [12].

The major challenge in the stacking approach is selecting the appropriate set of combination weights. The simplest methods are to combine all models by equal weight averaging or selecting the best base learner at each time sample. A more advanced method is to build a separate machine learning model, often referred to as the “metalearner” (or ensemble learner) [13], which learns to linearly combine base model predictions. However, the former basic approaches have been repeatedly found to outperform sophisticated weight combination methods in several competitions, which is known in the literature as the “forecast combination puzzle” [14]. Hence, we tackle the metalearning problem of selecting the optimal combination weights at each time sample based on a data-specific side-information vector.

The metalearning problem has been extensively studied in the literature based on two common approaches. In the first method, the base model predictions are fed to the ensemble learner, which is often selected to be artificial neural network (ANN) [15], support vector regressor (SVR) [16], linear regressor [17], etc. Here, the ensemble learner directly predicts the target sequence,

Manuscript received 19 May 2023; revised 2 August 2023; accepted 20 September 2023. Date of publication 26 September 2023; date of current version 14 May 2024. This paper was recommended for publication by Associate Editor Manuel Roveri upon evaluation of the reviewers’ comments. (Corresponding author: Arda Fazla.)

The authors are with the Department of Electrical and Electronics Engineering, Bilkent University, 06800 Ankara, Turkey (e-mail: arda@ee.bilkent.edu.tr; enesa@ee.bilkent.edu.tr; kozat@ee.bilkent.edu.tr).

Digital Object Identifier 10.1109/TAI.2023.3319308

whereas the predictions of the base learners are directly fed to the metalearner. This approach has well-known shortcomings. For instance, the ensemble learner often generates nonoptimal predictions due to a lack of proper context for these predictions. In other words, some base models may perform better than others in certain contexts or states. However, this context-specific information tends to get lost during the ensembling process, as the ensemble learner does not have access to this context information. Moreover, even when using other features to provide the context information as inputs to the ensemble learner, in addition to the predictions of the base learners, e.g., those extracted from the time series, the base predictions themselves tend to dominate the other features, i.e., the context information is hardly utilized [13]. The weights in such combinations, therefore, tend to be biased toward the prediction of the “best” base predictor, which causes the metalearner to overfit the said base model.

In contrast, the latter approach [11], [18] is based on solving a weight optimization problem that minimizes a certain loss function and learns to predict optimal combination weights based on the base model predictions and a data-specific side-information vector. This method bears in itself the structure to eliminate the well-known overfitting problem in the former direct combination approach as the ensemble feature vector does not contain the base model predictions. In the optimal setting, the feature vector should only carry information related to the target sequence (e.g., past values of the target sequence and external side-information related to the target sequence.); hence, the metalearner can learn the relation between the side-information vector and the base model predictions without overfitting to a single model [11]. However, in the context of the contemporary literature on the given approach, the ensemble learner is usually trained on multiple time series, where the ensemble model learns a single combination weight vector for each time series [11], [18]. However, in most real-life scenarios, time series data may not be the output of a single stochastic process since the environment can have changing characteristics [19]. In particular, the dynamics of the underlying system that generates the target sequence can exhibit temporally varying statistics [19]. Even though these properties can be examined by using a time-varying feature vector, the variance information is lost in the combination stage of the state-of-the-art ensemble algorithms in the literature [11], [18], as only a single weight vector is generated and used by the ensemble learner for each time series. To address these issues, we tackle the weight optimization problem in a time-varying manner. We output the combination weight vector at each time sample, dependent on the side-information vector, i.e., based on the context. Moreover, our ensembling approach can, therefore, learn which base models can work in which state or context better than the others, an important aspect of the model combination problem that was previously neglected in the literature.

Another essential factor in finding the optimal linear combinations of the base model predictions is the constraint space of the ensembling weight vector. In the optimal case where the statistics of the combined predictions and the target signal are fully known, naturally, the unconstrained combination of the base predictors achieves the lowest possible error [20]. In practice,

however, those statistics are rarely known, and the unconstrained combinations are subject to the learning problems, i.e., whether all the parameters will be correctly learned or not, and therefore not guaranteed to reach an optimal solution. In fact, since the parameter space is the entire Euclidean space of the appropriate dimension, i.e., the number of base learners, unconstrained combinations might lead to overfitting to a nonoptimal local minimum in the optimization problem [21]. Moreover, most recent work on ensemble learning in the literature focuses on complex models, such as deep neural networks [22], [23], which is, in fact, prone to overfitting to nonoptimal solutions due to the large number of parameters to be optimized in the model structure [22]. Hence, the common approach in many studies [11], [18], [24] is to constrain the weight combination problem to the convex case, where all the base prediction weights sum up to 1 and are nonnegative. Contrarily, in our work, we propose an optimization scheme that depends on both time and side-information (or context) for the first time. We demonstrate that our models can find the optimal solutions to the linear combination problem under all convex, affine (where weights sum up to 1), and unconstrained weight spaces in real-life datasets. In addition, we also theoretically analyze the weight combination problem under known statistics for all three constraints.

To this end, we propose two novel metalearners with the appropriate learning tasks, where we effectively combine the base predictions in a context and time-dependent manner. For the linear combination of the base predictions, our ensemble learners output a weight vector at each time sample, which is either unconstrained or amenable to satisfy the affine or convex constraints. Moreover, our ensemble learners use a superset of features in training, which equals to or extends the concatenated feature sets of the base models and does not include the base learner predictions themselves as the features of the ensemble learner. Our motivation here is to use the superset feature vector in order to make the combining model “context-aware” and less prone to overfitting to a prominent base model while exploring various constraint spaces. In particular, we employ the LightGBM (a gradient boosting machine [25]) and an ANN [26] as the metalearners to show the efficacy of our ensembling approach. Our ensembling approach is generic such that any differentiable machine learning model can be employed instead, given the appropriate learning task. In addition, we employ a novel training scheme for the metalearners, where we alleviate the possible overfitting problem of the ensemble learners to a specific base model.

B. Related Work

Ensemble models are heavily investigated in machine learning, computational learning theory, statistics, and signal processing [13], [27], [28], since they provide superior performance compared with their individual counterparts, provided that the individual learners are accurate enough and diverse [4], [5]. Common strategies for ensembling include boosting [8], bagging [9], and stacking [10]. We are interested in a special case of stacking called metalearning, where a separate machine learning model is trained to linearly combine base model predictions.

There have been simpler approaches to the model combination problem, such as training an ensemble model with the base model predictions as the feature vector without solving a separate optimization problem. For example, an SVR has been employed in the given fashion to predict electricity load demand [16]. However, these methods are prone to overfit to the base model predictions as the input vector lacks any side-information related to the target sequence. Although supplementary features can be used to provide context information as inputs to the ensemble learner, the predictions from the base models tend to overshadow these features, i.e., the context information is hardly utilized [13].

On the contrary, the problem of finding the optimal combination weights to linearly combine base model predictions has gathered significant attention in contemporary literature. More precisely, recent works have focused on solving a weight optimization problem that minimizes a certain loss function and learns to predict optimal combination weights [11], [18]. These studies differ in multiple premises, such as the ensemble model to be employed, the optimization problem to be solved, the structure of the feature vector to be fed to the ensemble learner, and the constraint on the linear combination space. In the given context, a super-ensemble model [18] has been proposed for daily streamflow forecasting, where the ensemble learner is a convex weighted combination of multiple machine learning algorithms. The weights are estimated by minimizing the root-mean-squared error (RMSE) to be produced by the ensemble prediction, which is the weighted linear combination of the base model predictions. The setback with the introduced approach is that the combination weights are set the same for all the test samples. Hence, this proposition does not consider the possible interchanging dynamics of the target sequence. However, real-life datasets often contain interchanging dynamics [19]. Therefore, some base models can work in certain time samples (or states) better than others, which are neglected in the ensembling process.

Another recent ensemble model is the FFORMA model [11], which is introduced in the M4 Forecasting Competition [11]. Here, the authors employ the XGBoost [29] as the metalearner, which learns to produce combination weights by minimizing the weighted average of the base model losses in a convex optimization constraint. The authors attract attention to the fact that they employ the target-specific side-information vector as the input to their ensemble model. Hence, the model learns to produce combination weights dependent on the side-information vector. However, in the given method, the ensemble learner is trained on multiple time series with similar characteristics, all strongly dependent on the environmental factors aggregated into the side-information vector. In the end, the ensemble model learns to predict the optimal combination weights for each time series, independent of time. Hence, the given model does not consider the data's time-varying characteristics while producing combination weights, such as the seasonality and trend of the target sequence [6]. In addition, the linear combination search space is only considered under a convex constraint. However, affine and/or unconstrained combinations can also lead to optimal results in real-life scenarios, as we illustrate in our simulations.

C. Contributions

Our main contributions to the literature can be summarized as follows.

- 1) We find the optimal combination weights of the base learners under three different weight constraints. To the best of authors' knowledge, for the first time in the literature, we solve the problem both in a time-varying and side-information-dependent context, where at each time sample, the optimal combination weights are found based on the target-specific side-information vector.
- 2) To learn the optimal time and side-information dependent weights, we present two novel and generic ensemble algorithms based on boosted decision trees [8] and neural networks [26], where we derive all the related equations. The introduced novel ensemble training scheme alleviates any training-related issues by the base models, such as colinearity, high correlation, and overfitting.
- 3) We compare our ensemble framework with the state-of-the-art machine learning models in the literature, as well as conventional ensembling approaches, in order to illustrate the superiority of our approach. We showcase our results in various well-known benchmark time series datasets [7], [30].
- 4) We publicly share the implementation of our algorithm for both model design, comparisons, and experimental reproducibility.¹

D. Organization

The rest of this article is organized as follows. In Section II, we introduce the problem of finding the time-dependent optimal base model combination weights based on the side-information vector under different constraints. In Section III, we present the training structure of our novel ensemble framework, based on minimizing a certain loss function to find the optimal linear combination weights. We then introduce two novel and generic ensemble learners in Section IV, and illustrate the performance of our algorithms via extensive experiments involving real-life and synthetic data in Section V. Finally, Section VI concludes this article. In addition, we analyze the associated learning costs of the optimal combination weight vectors under all three constraints in Appendix A, and provide the closed-form backpropagation equations for our ensemble algorithms in Appendix C.

II. PROBLEM STATEMENT

In this article, all vectors are real column vectors and are presented in boldface lowercase letters. Matrices are denoted by boldface uppercase letters. $x^{(k)}$ and $x_t^{(k)}$ denote the k th element of the vectors $\mathbf{x}$ and $\mathbf{x}_t$, respectively. $\mathbf{x}^T$ represents the ordinary transpose of $\mathbf{x}$. $X_{i,j}$ represents the entry at the i th row and the j th column of the matrix $\mathbf{X}$.

We study the prediction of time series data using a linear mixture of different learning models. The main sequence we

¹[Online]. Available: <https://github.com/ardafazla/context-time-aware-ensemble>

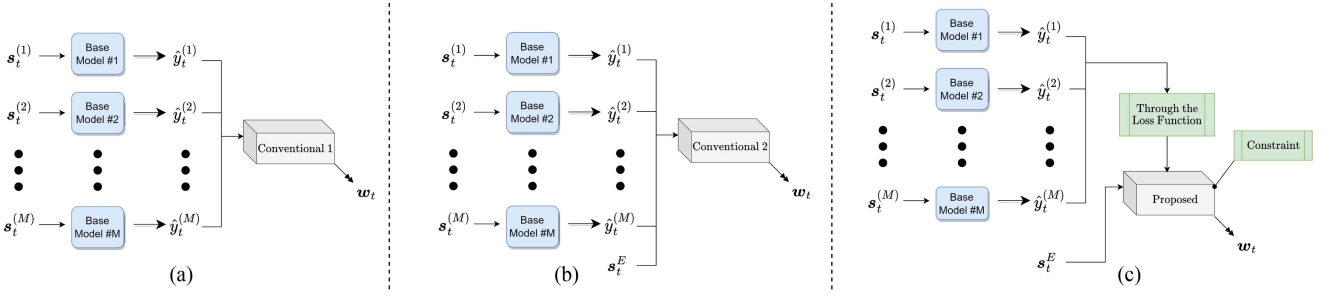


Fig. 1. Comparison of our ensembling approach and the conventional models in the literature. (a) Base model predictions are fed to the ensemble learner as the feature vector. (b) In addition, a data-specific side-information vector is fed to the ensemble learner. In both conventional models, the errors of the base models are not utilized, as the base model predictions are employed solely as features to the ensemble learner. On the contrary, in our approach (c), we do not feed the base model predictions to the ensemble learner as part of the feature vector. Instead, we utilize the errors of the base models through the generic ensemble loss function. Here, based on the given combination constraint, our model learns to produce the optimal combination vector at each time sample t .

aim to predict is $\{y_t\}$; to this end, we employ M machine learning models, called base models, each of which uses a side-information sequence $\{s_k^{(i)}\}$ for $k \leq t$ and $i = 1, \dots, M$. Here, the side-information sequence refers to the input vector that the model is fed with at each time sample. We emphasize that each base model is free to use a different side-information vector, e.g., $\{s_k^{(i)}\}$ could include the observed past information $\{y_k\}$ along with model-dependent features. At each time t , all the base models produce predictions $\hat{y}_t^{(i)}$. We combine these M predictions using a linear scheme such that the ensemble prediction $\hat{y}_t^E$ of y_t is given as

$$\hat{y}_t^E = w_t^T \hat{\mathbf{y}}_t \quad (1)$$

where $\hat{\mathbf{y}}_t = [\hat{y}_t^{(1)}, \dots, \hat{y}_t^{(M)}]^T$ is the base prediction vector of size M consisting of the individual scalar predictions of the base models and $w_t \in \mathbb{R}^M$ is the ensembling weight vector at time t . We note that even though the ensembling scheme is linear, the adaptive learning procedure that produces w_t could be highly nonlinear, as we show in Section III and Appendix A. When we observe y_t , we suffer the loss

$$\begin{aligned} L_t &= \ell(y_t, \hat{y}_t^E) = \ell(y_t, w_t^T \hat{\mathbf{y}}_t) \\ &= \ell\left(y_t, \sum_{i=1}^M w_t^{(i)} \hat{y}_t^{(i)}\right) \end{aligned} \quad (2)$$

where ℓ is a specific loss function of choice, such as the squared error loss.

Conventionally, in the literature, the learning of the weight vector w_t depends only on the base models' predictions, i.e., $w_t = f(\hat{\mathbf{y}}_t)$ for some function f representing a learning model [31], [32]. Here, the ensemble learner directly predicts the target sequence, given the base model predictions as the input vector at time t , as illustrated in Fig. 1(a). In the given setting, the ensemble learner tends to produce nonoptimal predictions as it lacks the proper context in which these predictions are produced. For example, some base models may perform better than others in specific contexts or states, but this information is lost during the ensembling process because the metalearner is unaware of this context of information [13]. Moreover, even

if additional context information is employed as inputs to the ensemble learner, as shown in Fig. 1(b), the base predictions themselves tend to dominate the other features, i.e., the context information is hardly utilized [13]. As a result, the weights in these combinations frequently favor the prediction of the "best" base predictor, which leads the metalearner to overfit to the aforementioned base model.

Therefore, we introduce a novel ensembling scheme in which the metalearner solves a weight optimization problem to minimize a specific loss function and learns to predict optimal combination weights based on a data-specific side-information vector. In this approach, the base models' predictions are not directly incorporated into the feature vector of the base model. Instead, they are utilized through the minimization of a certain loss function, as illustrated in Fig. 1(c). In our proposed setting, we investigate three different mixture approaches, which are identified by how they constrain the combining weight vector $w_{s_t^E}$; these are the unconstrained linear combination where each $w_{s_t^E}^{(i)}$ is free to take any real value, the affine linear combination where the components of the weight vector sum up to 1, i.e., $w_{s_t^E}^T \mathbf{1} = 1$ and the convex linear combination where the weight vector not only has its components sum to 1 but also all nonnegative, i.e., $w_{s_t^E}^T \mathbf{1} = 1$ and $w_{s_t^E}^{(i)} \geq 0 \forall i$; all of these constraints apply to all time steps. Hence, we aim to adaptively find the "best" weights that linearly combine the predictions of M parallel running base models to minimize a given loss ℓ at each time step t using the ensemble side-information vector s_t^E under three different constraints imposed on the weight vector.

To illustrate the ensembling process and the use of the side-information vector, let us take the example of hourly wind power nowcasting and employ two parallel-running base learners. The first model might utilize three features: wind power, wind speed, and wind direction from the previous hour. The second model could use two features: the wind power and direction from 24 h ago, as the side-information. The ensemble model could then incorporate all features used by the base learners, along with an hour-of-day indicator to recognize the learning patterns of each base learner. Therefore, "side-information" refers to the data and model-specific input vector that each model is fed with at each time sample, which can vary based on the distinct

learning architectures of the models. We emphasize that during the ensembling process, the side-information, or the input vector, of the ensemble learners does not include the predictions of the base learners. This prevents our ensemble learners from overfitting to the predictions of a single base model, which we show in our simulations.

In the optimal case where the statistics of the combined predictions and the target sequence are fully known, naturally, the unconstrained combination of the base predictors achieves the lowest possible error [20]. However, in practice, those statistics are rarely known. Hence, finding the optimal linear combination weights through a metalearner is subject to learning problems, i.e., whether all the parameters will be correctly learned or not, and therefore not guaranteed to reach an optimal solution. Furthermore, in the unconstrained case, there is no restriction on the linear combination search space, meaning the associated learning problem is larger compared with when the search space is limited to a constraint-specific subspace; hence, unconstrained combinations may overfit or may not have enough data to find the best optimal combination leading to a suboptimal solution. As we decrease the size of the linear combination search space through constraints, the learning complexity decreases. However, the chance of excluding the optimal combination weight vector in the subspace increases accordingly, which is the well-known bias-variance tradeoff in the machine learning literature. Therefore, we consider three distinct weight constraints, namely, the unconstrained, affine and convex cases that differ on the complexity of the associated learning problem. Further, analytical overview of the intuition behind employing an ensemble learner for the given constraints is presented in Appendix A.

Remark: It should be noted that the metalearning approach, which is based on solving a weight optimization problem to minimize a particular loss function and predict optimal combination weights using both base model predictions and a data-specific side-information vector, has been employed multiple times in the literature [11], [18]. However, in the context of existing research on this approach, the ensemble learner is typically trained on multiple time series. In this case, the ensemble model learns a single optimal combination weight vector for each time series, independent of the time sample t . Hence, the produced combination weights are independent of time. In contrast, our approach examines a single time series and addresses the weight optimization problem in a time-varying manner. We output the combination weight vector at each time sample t , based on the side-information vector and the current context. This approach allows the combination weights to be time-dependent and to change dynamically based on the context.

Next, we present the learning and training scheme of our novel ensembling approach, as well as the generic ensemble algorithms used to find the optimal combination weight vectors, subject to the given constraint.

III. INTRODUCED APPROACH

In order to linearly combine the predictions of different base models by assigning weights to each model, we use ensemble algorithms. Our ensemble algorithms learn to combine the

predictions of different base algorithms by minimizing the loss function given in (2), where y_t is the observed sequence at time t , ℓ is a given differentiable loss function, such as the squared error loss, $\hat{y}_t^E$ is the prediction of the ensemble algorithm, M is the number of base models, $\hat{y}_t^{(i)}$ are the predictions of the base models, and $w_{s_t^E}^{(i)}$ are the weights assigned to the base models, by utilizing the side-information sequence s_t^E .

In order to train our ensemble algorithms, we introduce a novel ensemble training scheme, illustrated in Fig. 2. Our training scheme consists of two phases: the offline phase and the online phase. In the offline phase, first, the base models are independently trained on a certain partition of the training dataset, given as $1 \leq t \leq T_1$. Then, the predictions of the base models are obtained on the remaining partition of the training data, given as $T_1 + 1 \leq t \leq T$, where these predictions are then used to train the ensemble algorithm. Hence, as the predictions are gathered on unseen data, we overcome the possible overfitting issues of the base models, which is a well-known problem for the conventional ensembling approaches [33]. Each base model i makes predictions based on a side-information vector $s_t^{(i)}$, which may consist of the past observations of the sequence y_t along with some additional related information specific to the observed sequence or the base model. Our ensemble model is trained with the errors of the base models while employing the side-information sequence s_t^E , which may consist of the past observations of the sequence y_t , side-information related to the base models, and some additional information specific to the observed sequence.

In the online phase, we use our pretrained ensemble algorithm from the offline phase to produce combination weights for the base algorithms on the new data, which, in our case, are the new observations following the training samples of the sequence y_t . First, we retrain the base models on the whole training dataset, given as $1 \leq t \leq T$. Then, we produce the base model predictions for the test dataset, $T + 1 \leq t \leq T_2$. We then update the ensemble side-information vector s_t^E for the test duration. Finally, we produce the ensemble prediction $\hat{y}_t^E = \sum_{i=1}^M w_{s_t^E}^{(i)} \hat{y}_t^{(i)}$, for $T + 1 \leq t \leq T_2$. We provide a detailed view of our ensemble learning procedure in Appendix B.

We next introduce two novel and generic ensemble algorithms to achieve the optimal weights under the three combination constraints introduced in Section II.

IV. ENSEMBLE MODELS

A. LightGBM Ensemble

Gradient boosting decision trees (GBDT) are sequentially trained decision tree ensembles. At each iteration, GBDT minimizes the residual error made by the previous trees in the sequence. LightGBM is a state-of-the-art GBDT model that deviates from finding split points to minimize a certain loss function for the whole data during training and is efficient in terms of memory consumption and training speed compared with its counterparts [25].

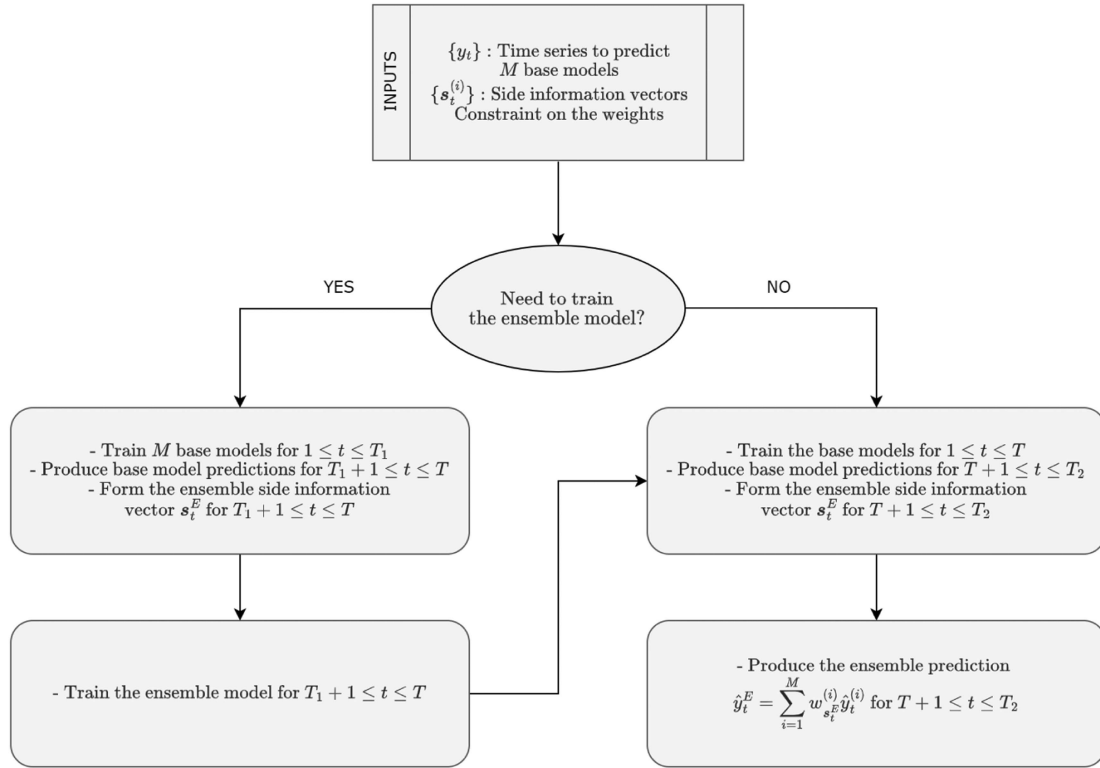


Fig. 2. Flowchart of the ensemble training scheme.

We employ LightGBM to search for the optimal ensemble combination weights, as described in Algorithm 1. Here, LightGBM requires the gradient and Hessian of the objective function (2) in order to minimize the loss with respect to the combined weight vector $w_{s_t^E}$. We study the linear combination of the base predictions under unconstrained, convex constrained, and affine constrained conditions, as explained in Section II. For each base learner i , LightGBM produces an output value $p_{s_t^E}^{(i)}$, where we then apply the necessary transformation to achieve $w_{s_t^E}^{(i)}$ under the given weight constraint. Therefore, for all three cases, we study and provide the gradient and Hessian of (2) with respect to $p_{s_t^E}^{(i)}$, for each base learner i . For all constraints, we set the loss function ℓ as the squared error loss, although any differentiable loss function can be selected, and assume that we have the base predictions for $1 \leq t \leq T$, as explained in Fig. 2. Therefore, at any given time t from the training dataset, the loss to be minimized by the LightGBM ensemble is

$$L_t = \ell(y_t, \hat{y}_t^E) = (y_t - \hat{y}_t^E)^2 = \left(y_t - \sum_{i=1}^M w_{s_t^E}^{(i)} \hat{y}_t^{(i)} \right)^2$$

where y_t is the observed sequence, and $\hat{y}_t^E$ is the prediction of the ensemble algorithm, at time t . The transformation from $p_{s_t^E}^{(i)}$ to $w_{s_t^E}^{(i)}$ is given as $w_{s_t^E}^{(i)} = \tau(p_{s_t^E}^{(i)})$, where τ indicates the transformation operation and is specific to the given weight constraint. Hence, the gradient and Hessian to be calculated at

any time t , for the base learner i , is given as

$$G_i = \frac{\partial L_t}{\partial p_{s_t^E}^{(i)}}, H_i = \frac{\partial^2 G_i}{\partial^2 p_{s_t^E}^{(i)}}. \quad (3)$$

In Appendix C-A, we explain the necessary transformations and provide the gradient and Hessian calculations for all three constraints of the LightGBM ensemble model.

B. Multilayer Perceptron (MLP) Ensemble

ANN are models based on the brain cells' processing structure and are used to model complex patterns and problems [34]. The learning structure of the model is based on interconnected nodes that are organized into layers, which are utilized to transmit information through weighted connections. As our ensemble algorithm, we employ a certain class of ANN called MLP, which is an ANN with more than two layers. MLPs are heavily used in machine learning due to their ability to learn complex and nonlinear relations with high training speed compared with their counterparts [35].

As a specific example, we provide the equations for a two-layered network architecture consisting of an input layer, a hidden layer, and an output layer. For more layers, naturally, our derivations can be straightforwardly extended. At the end of the output layer, we apply a transformation operation that maps the weights of the ensemble algorithm according to the given weight constraint. We study the linear combination of the base predictions under affine unconstrained, affine constrained, and convex constrained conditions. The feedforward equations of

our model are given as

$$\begin{aligned} \mathbf{v} &= \mathbf{U}^{(1)} \mathbf{x} \\ \mathbf{z} &= f(\mathbf{v}) \\ \mathbf{p} &= \mathbf{U}^{(2)} \mathbf{z} \\ \mathbf{w} &= \tau(\mathbf{p}) \end{aligned}$$

where $\mathbf{x} \in \mathbb{R}^K$ is the input vector, $\mathbf{U}^{(1)} \in \mathbb{R}^{L \times K}$ are the layer weights connecting the input layer to the hidden layer, $\mathbf{z} \in \mathbb{R}^L$ is the input of the hidden layer, which is the rectified linear activation function (ReLU) f applied to $\mathbf{v} \in \mathbb{R}^L$. $\mathbf{U}^{(2)} \in \mathbb{R}^{M \times L}$ are the layer weights connecting the hidden layer to the output layer, $\mathbf{p} \in \mathbb{R}^M$ is the output vector of the model, and $\mathbf{w} \in \mathbb{R}^M$ is the weight vector for the linear combination of the base models, obtained by applying the transformation operation τ to $\mathbf{p}$. K , L , and M are the lengths of the input, hidden, and output layers, respectively.

Our ensemble algorithm learns to produce linear combination weights by minimizing the loss function (2). For all constraints, we set ℓ as the squared error loss, although any differentiable loss function can be selected, and assume that we have the base predictions for $1 \leq t \leq T$, as explained in Fig. 2. Therefore, at any given time t from the training data, the loss to be minimized by the MLP ensemble is given as

$$L_t = \ell(y_t, \hat{y}_t^E) = (y_t - \hat{y}_t^E)^2 = \left(y_t - \sum_{i=1}^M w_{s_t^E}^{(i)} \hat{y}_t^{(i)} \right)^2$$

where y_t is the observed sequence, and $\hat{y}_t^E$ is the prediction of the ensemble algorithm, at time t . Here, we employ the notations $x_i \triangleq x_{s_t^E}^{(i)}$, $w_i \triangleq w_{s_t^E}^{(i)}$, and $p_i \triangleq p_{s_t^E}^{(i)}$, for simplicity. The loss is minimized by iteratively updating the layer weights $\mathbf{U}^{(1)}$ and $\mathbf{U}^{(2)}$ with backpropagation. The backpropagation equations are given as

$$\begin{aligned} \frac{\partial L_t}{\partial U_{l,k}^{(1)}} &= \sum_{j=1}^M \sum_{i=1}^M \frac{\partial L_t}{\partial w_i} \frac{\partial w_i}{\partial p_j} \frac{\partial p_j}{\partial z_l} \frac{\partial z_l}{\partial v_l} \frac{\partial v_l}{\partial U_{l,k}^{(1)}} \\ \frac{\partial L_t}{\partial U_{m,l}^{(2)}} &= \sum_{i=1}^M \frac{\partial L_t}{\partial w_i} \frac{\partial w_i}{\partial p_m} \frac{\partial p_m}{\partial U_{m,l}^{(2)}} \end{aligned}$$

where $1 \leq l \leq L$, $1 \leq k \leq K$, $1 \leq m \leq M$. We then update $\mathbf{U}^{(1)}$ and $\mathbf{U}^{(2)}$ using stochastic gradient descent (SGD). The update equations are given as

$$U_{l,k}^{(1)} = U_{l,k}^{(1)} - \alpha \frac{\partial L_t}{\partial U_{l,k}^{(1)}}, \quad U_{m,l}^{(2)} = U_{m,l}^{(2)} - \alpha \frac{\partial L_t}{\partial U_{m,l}^{(2)}}$$

where α is the learning rate hyperparameter of SGD. In Appendix C-B, we explain the necessary transformations and provide the closed-form backpropagation equations for all three constraints of the MLP ensemble model.

We now have the update equations to train both the MLP ensemble and LightGBM ensemble algorithms according to Fig. 2.

Remark: Note that our motivation for introducing two novel ensemble learning algorithms comes from the distinct learning

structures of both models. The LightGBM ensemble determines the optimal linear combination weights by finding the best splitting points for the data samples. This is achieved by using the gradient and Hessian of the loss function with respect to the base model outputs, as demonstrated in (3). In contrast, the MLP ensemble only uses the gradient of the loss function with respect to the base model outputs while optimizing the weights of its layers. Thus, depending on the complexity of the data and the weight optimization problem, the results of our metalearners may vary, as demonstrated in Section V. In addition, although we provide two different models to use as our ensemble learners, any machine learning model, such as linear regression models [36] etc., can be employed provided that they support a custom differentiable loss for minimization.

In the following section, we demonstrate the performance of our ensemble models under different constraints, using a variety of datasets.

V. EXPERIMENTS

In this section, we evaluate our ensemble learners, which are the LightGBM ensemble in Section IV-A and the MLP ensemble in Section IV-B, on various time series datasets. First, we describe the datasets, models and benchmarks, hyperparameter tuning and optimization methods, and error metrics used in our experiments. Then, we provide a detailed analysis of our results.

A. Datasets

In the first part of our simulations, we synthetically generate a dataset that heavily relies on a given side-information vector (context), and interchanges certain characteristics based on the context and time. Here, we demonstrate that our ensemble learners are able to fully utilize the base learners to capture all of the patterns present in the data. Next, we use three well-known benchmark time series datasets that show multiple patterns/seasonalities and are dependent on a data-specific side-information vector, to showcase the performance of our models.

Following is a brief description of the real-life datasets.

1) *Turkish Natural Gas Demand:* Daily data of the total residential natural gas demand in Turkey between years 2018 and 2020. The dataset is 1000 samples long, and shows weekly and monthly seasonality patterns. There are a total of 58 features, which can be utilized within the side-information vectors, i.e., the input vectors to the models. These features include past observations of the total residential natural gas demand, weather-related factors, such as temperature and wind speed, along with categorical features, such as day-of-week, day-of-month, and day-of-year indicators.

2) *Istanbul Stock Exchange:* Daily returns of the Istanbul Stock Exchange [30] with seven other international indexes and the past observations of the target sequence given as features, which can be utilized within the side-information vectors of the models. The dataset captures the period between 5 June, 2009, and 22 February, 2011. The dataset was obtained from the UCI Machine Learning Repository [37].

TABLE I
STATISTICS OF THE REAL-LIFE DATASETS

Dataset	Length	Seasonality	Feature Size
Natural Gas Demand	1000	[7, 30.5]	58
Istanbul Stock Exchange	536	[5, 20]	23
M5 Forecasting	1941	[7]	56

3) *M5 Forecasting*: Daily data of unit sales of products sold in the USA by the retail company Walmart [7]. There are a total of 3049 products, which are classified into three different categories and seven different departments. The products are sold in ten different Walmart stores in three different states. Here, we fit the data to the total amount of products sold in a randomly chosen category in a randomly chosen Walmart store. There are a total of 56 features that include the past observations of the products sold and categorical features, such as binary indicators for holidays. The data show weekly seasonality patterns with strong nonlinearity in certain regions.

Table I summarizes the statistics of the datasets used in our experiments. Note that each dataset is split into 60% training, 20% validation, and 20% test. All of the features explained in the datasets can be utilized withing the side-information vectors (inputs) of the models used in the experiments. For each model, we use statistical tests and model-specific attributes, such as feature importance graphs to find the best feature subspace.

B. Models

As our base learners, we employ four models from different backgrounds to satisfy diversity in our ensemble structure, which are SARIMAX [38], LightGBM [25], Naïve, and MLP. We then train our two ensemble models, namely, the LightGBM ensemble and the MLP ensemble, to linearly combine the predictions of these base learners. Next, we compare the performance of our ensemble learners against a collection of current state-of-the-art techniques in forecasting with multiple patterns. To that extent, we include recurrent neural network (RNN) [39], long short-term memory (LSTM) [40], gated recurrent unit (GRU) [41], trigonometric seasonality, box-cox transformation, ARMA errors, trend, and seasonal components (TBATS) [42], MLP [34], and two MLP metalearners optimized following the conventional methods given in Fig. 1. Note that all of the aforementioned models employ a model-specific side-information vector as the input. During the training of our ensemble learners, we set the constraints on the linear optimization weights as hyperparameters along with the number of base learners. Thus, two additional hyperparameters are initiated with training our ensemble learners alongside the parameters of the employed model. We further explain the structures of the models in Table II. For the rest of the simulations, we refer to these models with their names given in Table II in order to reduce repetition.

C. Hyper Parameter Tuning and Optimization

In our simulations, we first search for the optimal hyperparameters during the model training process, which minimizes the total squared error loss on the validation set. Once the optimal

TABLE II
ILLUSTRATIONS OF THE MODELS USED IN OUR SIMULATIONS

Model Name	Explanation
SARIMAX	Statistical forecasting model with seasonality and external variables, introduced by Box and Jenkins [39]
LightGBM	Tree-based gradient boosting model
Naïve	Prediction of the future value based on the most recent observation
MLP	ANN with multiple layers
RNN	Recurrent Neural Network introduced in [39].
LSTM	Long short-term memory introduced in [40].
GRU	Gated Recurrent unit introduced in [41].
LightGBM Ensemble	Ensemble learner introduced in Section IV-A
MLP Ensemble	Ensemble learner introduced in Section IV-B
TBATS	Exponential smoothing state space model that utilizes ARIMA errors, Box-Cox transformation, Trend and Seasonal components
Conventional 1	MLP meta-learner trained by the conventional approach illustrated in Fig. 1 (a)
Conventional 2	MLP meta-learner trained by the conventional approach illustrated in Fig. 1 (b)

hyperparameters are determined, we retrain the models using the best hyperparameter set on the entire training and validation sets. Note that the hyperparameter tuning for the SARIMAX and the TBATS models do not employ a validation structure, as they are based on minimizing a certain information criterion. Subsequently, after the base models are trained on the given dataset, we then train our ensemble models and the conventional ensembles. Note that the conventional MLP ensemble and the base MLP model share the same search space. On the other hand, our MLP and LightGBM ensembles also share the hyperparameter search space of their base model counterparts; however, the given constraint and the set of base learners are additionally searched. The training of our ensemble models is done, as illustrated in Fig. 2. The search space for the hyperparameters of the models employed in our simulations is provided in Table III.

D. Error Metrics

We compare the accuracy of the models used in our simulations based on different error metrics given as mean squared error (mse), RMSE, and mean absolute error (MAE). We only consider one-step-ahead forecasting, and during learning, all models are optimized to minimize the squared error loss function given in (2). During the validation of the models, we select the best set of hyperparameters based on the MSE on the validation set.

E. Synthetic Data Experiments

In order to verify the capability of our ensembling scheme to detect all different regions in the data by exploiting the side-information in a time-dependent manner, we conduct our initial experiments on synthetic data. The synthetic data consist of two manually generated and independent data components that are generated as

$$y_t^{\{1\}} = -0.5y_{t-1}^{\{1\}} + 0.4y_{t-2}^{\{1\}} + v_t^{\{1\}}$$

TABLE III
HYPERPARAMETER SEARCH SPACE

Model Name	Search Space
SARIMAX	$p:\{0,1,2,3\}$, $d:\{0,1\}$, $q:\{0,1,2,3\}$ $P:\{0,1,2\}$, $D:\{0,1\}$, $Q:\{0,1,2\}$
LightGBM	$n_estimators:\{100:2500\}$ $learning_rate:\{0.001:0.1\}$ $num_leaves:\{7:255\}$ $max_depth:\{3:7\}$ $min_child_samples:\{5:50\}$
MLP	$hidden_layer_neurons:\{8:128\}$ $activation:\{ "logistic", "tanh", "relu" \}$ $learning_rate:\{0.0001:0.1\}$ $max_iterations:\{100:1000\}$
RNN	$n_layers:\{1,2,3\}$ $learning_rate:\{0.0001:0.1\}$ $n_epochs:\{100:2000\}$ $dropout:\{0.0:0.5\}$ $batch_size:\{16:128\}$
LSTM	$n_layers:\{1,2,3\}$ $learning_rate:\{0.0001:0.1\}$ $n_epochs:\{100:2000\}$ $dropout:\{0.0:0.5\}$ $batch_size:\{16:128\}$
GRU	$n_layers:\{1,2,3\}$ $learning_rate:\{0.0001:0.1\}$ $n_epochs:\{100:2000\}$ $dropout:\{0.0:0.5\}$ $batch_size:\{16:128\}$
Our Ensemble	$constraint:\{ "unconstrained", "convex", "affine" \}$ $base_learner_set:\{ \text{all possible combinations of SARIMAX, LightGBM, MLP and Naïve} \}$
Conventional Ensemble	$base_learner_set:\{ \text{all possible combinations of SARIMAX, LightGBM, MLP and Naïve} \}$

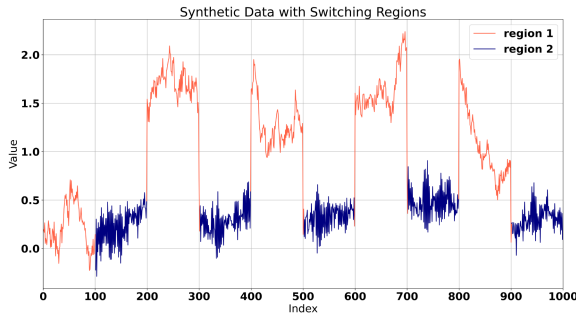


Fig. 3. Synthetic data with two distinguished regions, generated by a deterministic switching process.

$$y_t^{\{2\}} = y_{t-1}^{\{2\}} + v_t^{\{2\}}$$

where $v_t^{\{1\}}$ and $v_t^{\{2\}}$ are sample functions from a stationary white Gaussian process with unit variance. To this end, we generate synthetic data with a deterministic switching, with two regions, given by the following set of equations:

$$y_t = \begin{cases} 0.2y_t^{\{1\}} + 0.8y_t^{\{2\}} & (\text{mod } t, 200) < 100 \\ 0.8y_t^{\{1\}} + 0.2y_t^{\{2\}} & (\text{mod } t, 200) \geq 100. \end{cases}$$

Fig. 3 illustrates the generated synthetic data. The data follow two clearly distinguishable patterns/regions interchanging in time. Note that, as explained in Section III, our ensemble models take a data-specific side-information vector as the input and

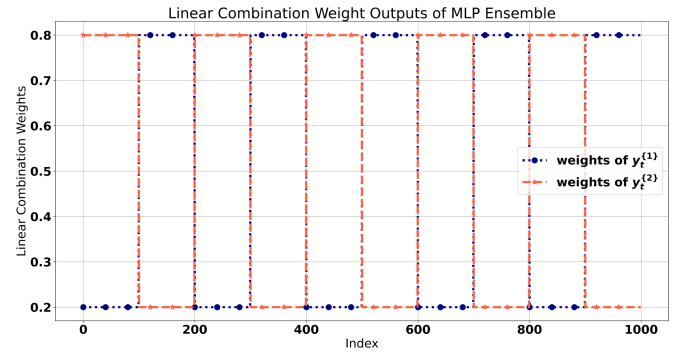


Fig. 4. Resulting linear combination weight outputs of the MLP ensemble under convex constraint. Here, the results for the training and test datasets are merged for visual integrity.

TABLE IV
COMPARISON OF ENSEMBLE MODELS ON THE SYNTHETIC DATASET

Model	MSE	RMSE	MAE
MLP ensemble	0.0	0.0	0.0
LightGBM ensemble	0.0	0.0	0.0
Conventional 1	0.18	0.43	0.41
Conventional 2	0.02	0.12	0.15

The best results for each metric are shown in bold.

linearly combine the base learner predictions according to the side-information vector to obtain a superior prediction. To this end, our two data components, $y_t^{\{1\}}$ and $y_t^{\{2\}}$, mimic the base model predictions that we would normally obtain for the real-life datasets. In addition, we provide the time information t as the side-information vector to our ensemble models. Hence, as we have claimed in Section II, our ensemble models should be able to predict the target sequence y_t by fully comprehending the switching regions given in Section V-E, and outputting the correct linear combination weights for the given data components (base learner predictions), accordingly.

The resulting linear combination weight vector on the whole dataset of the MLP ensemble under convex constraint is illustrated in Fig. 4. We also repeat the same experiments for the Conventional 1 and Conventional 2 ensemble models and compare the results in terms of the error metrics on the test dataset in Table IV. As clearly seen, the conventional ensembles fail to learn the switching regions present in the data, whereas our ensembling scheme perfectly fits the switching regimes, thus confirming the necessity of our ensemble structure. Our ensemble models can output the optimal linear combination weight vector at each time step, based on the available context information. On the other hand, conventional ensemble algorithms cannot find these switching regions accurately, as the context information is not fully utilized due to the model architecture. Note that both the MLP ensemble and the LightGBM ensemble fit the data perfectly, but we only illustrate the resulting weight combination vector of the MLP ensemble for simplicity.

TABLE V
RESULTS OF THE SIMULATIONS BASED ON REAL-LIFE DATASETS

Data Model\Metric	Natural Gas Demand			Istanbul Stock Exchange			M5 Forecasting		
	MSE(10^{11})	MAE(10^4)	RMSE(10^4)	MSE(10^{-5})	MAE(10^{-4})	RMSE(10^{-4})	MSE(10^3)	MAE	RMSE
MLP	47.13	119.64	217.09	21.43	121.78	146.4	46.889	154.65	216.54
SARIMAX	24.87	100.12	157.69	12.35	89.12	111.14	40.297	150.65	200.74
LightGBM	18.49	81.04	135.98	12.24	87.83	110.65	34.247	140.27	185.06
Naïve	53.3	126.88	230.88	33.43	143.5	182.83	213.997	351.93	462.6
TBATS	42.12	107.98	205.23	18.1	105.04	134.56	52.606	163.21	229.36
RNN	28.3	106.14	168.23	12.01	89.16	109.59	33.387	139.37	182.72
LSTM	30.19	106.91	173.76	12.3	88.66	110.91	32.37	137.05	179.91
GRU	30.54	97.78	174.75	12.31	88.15	110.96	33.096	137.67	181.92
MLP Ensemble	14.66	75.91	121.09	12.09	86.96	109.96	31.367	133.56	177.11
LightGBM Ensemble	12.21	75.5	114.95	11.55	87.12	107.47	30.014	131.49	173.25
Conventional 1	19.13	82.12	138.32	12.87	91.3	113.46	34.826	142.43	186.62
Conventional 2	18.86	81.66	137.33	15.27	96.48	123.56	36.424	148.64	190.85

The best results for each test dataset and metric are shown in bold. For our LightGBM and MLP ensembles, the ensemble weight constraint that achieves the best validation score has been selected independently for each dataset.

TABLE VI
COMPARISON OF OUR ENSEMBLE ALGORITHMS UNDER ALL THREE CONSTRAINTS

Data Model\Metric	Natural Gas Demand			Istanbul Stock Exchange			M5 Forecasting		
	MSE(10^{11})	MAE(10^4)	RMSE(10^4)	MSE(10^{-5})	MAE(10^{-4})	RMSE(10^{-4})	MSE(10^3)	MAE	RMSE
MLP Ens. Convex	14.66	75.91	121.09	12.12	87.66	110.09	31.544	136.59	177.61
MLP Ens. Affine	19.96	83.18	141.28	12.09	86.96	109.96	31.367	133.56	177.11
MLP Ens. Unconstrained	18.04	80.97	134.32	13.16	92.21	114.73	34.969	142.29	187
LightGBM Ens. Convex	13.21	75.5	114.95	12.09	86.87	109.95	30.014	131.49	173.25
LightGBM Ens. Affine	15.99	78.85	126.49	11.55	87.12	107.47	35.681	143.89	188.89

The best results for each test dataset and metric are shown in bold. Note that the LightGBM ensemble under unconstrained weight condition is not provided here, as the errors produced by the model are high compared with other models.

F. Real-Life Data Experiments

We evaluate the accuracy of our ensemble framework in comparison to the base models that we employ, conventional ensembling approaches and state-of-the-art forecasting techniques. Our results are demonstrated on three benchmark time series datasets that exhibit complex patterns and a strong context dependency. Our simulation results are displayed in Table V. In all three datasets employed in our experiments, our MLP and LightGBM ensembles surpass the performance of all other methods.

We highlight that in the Natural Gas Demand and M5 Forecasting datasets, the base models are capable of capturing distinct patterns in the data, thus outperforming other base models in specific regions. As a result, our ensemble models excel by learning to predict each region by emphasizing the importance of the corresponding base model in those areas. This effect is evident, as there is a significant difference between the performance of our model and the base models. In contrast, the Istanbul Stock Exchange dataset features a smaller feature vector compared with the other datasets, as given in Table I. Moreover, the time series data strongly depend on its recent observations, seen by the performance of the Naïve model. Consequently, while our ensemble models still outperform other models, the performance improvement compared with the base models is less significant. Furthermore, it is important to acknowledge that while we compare the performance of our ensemble models against the state-of-the-art forecasting models in the literature, these models could also be utilized as the base learners to our ensemble models. Thus, our ensemble models can be employed to boost the performance of the state-of-the-art forecasting

models, provided that the base learners in the ensemble framework are accurate enough and diverse [4], [5].

In Table VI, we present a comparative analysis of our LightGBM and MLP ensembles under all three constraints. It is important to note that the efficacy of these constraints is highly dependent on the specific dataset. For instance, in the Istanbul Stock Exchange dataset, both ensemble models achieve optimal results with the affine constraint. Conversely, in the Natural Gas Demand dataset, the convex constraint leads to superior performance for both ensembles when compared with other settings. Consequently, we deduce that the ideal weight combination constraint for our ensemble techniques is contingent upon the complexity, context, and size of the datasets. This is evident with our explanation in Section II and Appendix A, as all three constraints differ in terms of how they restrict the linear combination search space. Even though the unconstrained weight case should theoretically obtain the best results in the optimal learning case, the statistics of the target sequence are seldom truly known; hence, constraining the search space may lead to superior results, as shown in our simulations.

VI. CONCLUSION

We studied the problem of predicting sequential time series data where we combine the predictions of multiple diverse machine learning models using a novel ensemble framework. For the first time in the literature, we solve the problem of finding the optimal combination weights of the base learners both in a time-varying and context-dependent manner. To this end, we introduce two novel and generic ensemble algorithms to find the optimal combination weight vectors under unconstrained,

affine constrained, and convex constrained conditions. In addition, we introduce a novel training scheme for our ensemble models, which is centered around solving a weight optimization problem that minimizes a specific loss function. This approach also mitigates potential issues related to the training of meta-learners, such as colinearity, high correlation, and overfitting to the base algorithms. Through various experiments involving synthetic and well-known real-life sequential data, we demonstrate the significant performance gains achieved by our ensemble models compared with the base models used in the experiments, conventional ensembling methods, and state-of-the-art forecasting models in the literature.

APPENDIX A ANALYSIS ON CONSTRAINTS

Here, we analyze the optimal weight vectors for ensembling as well as the associated costs under known statistics for each of the three constraints imposed on the ensembling weight vector $\mathbf{w}_{s_t^E} \in \mathbb{R}^M$, where M is the number of base models in the ensemble. Let $\mathbf{C}(s_t)$ be the conditional autocorrelation matrix at time t of the base prediction vector $\hat{\mathbf{y}}_t = [\hat{y}_t^{(1)}, \dots, \hat{y}_t^{(M)}]^T$ given the superset of the side-information vectors used by all the base algorithms and specific to the ensemble, and let $\mathbf{a}(s_t)$ be the conditional cross-correlation vector at time t between the target signal y_t and the base prediction vector given the superset side-information, i.e.,

$$\begin{aligned}\mathbf{C}(s_t) &= \mathbb{E}[\hat{\mathbf{y}}_t \hat{\mathbf{y}}_t^T | s_t^E] \\ \mathbf{a}(s_t) &= \mathbb{E}[y_t \hat{\mathbf{y}}_t | s_t^E].\end{aligned}$$

In the unconstrained case, the components of $\mathbf{w}_{s_t^E}$ are free to take any real number; therefore, the corresponding optimization problem given s_t^E for a given differentiable loss function ℓ is

$$\min_{\mathbf{w}_{s_t^E} \in \mathbb{R}^M} \ell \left(y_t, \sum_{i=1}^M w_{s_t^E}^{(i)} \hat{y}_t^{(i)} \right) \quad (4)$$

where the feasible region is the entire M dimensional Euclidean space. If the statistics $\mathbf{C}(s_t)$ and $\mathbf{a}(s_t)$ are known at all times, the optimal unconstrained ensembling vector at time t under the expected squared error loss, i.e., when $\ell(y_t, \hat{y}_t) = \mathbb{E}[(y_t - \hat{y}_t)^2 | s_t^E] = \mathbb{E}[(y_t - \sum_{i=1}^M w_{s_t^E}^{(i)} \hat{y}_t^{(i)})^2 | s_t^E]$, is the solution to the normal equations and given as

$$\mathbf{w}_{s_t^E, \text{unc}}^* = \mathbf{C}(s_t)^{-1} \mathbf{a}(s_t).$$

Furthermore, assuming the target signal has the conditional variance $\sigma(s_t)^2$ at time t , the conditional mse of the unconstrained ensembling is given by

$$L_{t, \text{unc}}^* := \sigma(s_t)^2 - \mathbf{a}(s_t)^T \mathbf{C}(s_t)^{-1} \mathbf{a}(s_t).$$

In the affine-constrained case, the weight vector components are required to sum up to 1. This translates to the optimization

problem given as

$$\begin{aligned}\min_{\mathbf{w}_{s_t^E} \in \mathbb{R}^M} \quad & \ell \left(y_t, \sum_{i=1}^M w_{s_t^E}^{(i)} \hat{y}_t^{(i)} \right) \\ \text{subject to} \quad & \mathbf{w}_{s_t^E}^T \mathbf{1} = 1\end{aligned} \quad (5)$$

where the feasible region is now an $M - 1$ -dimensional hyperplane in $\mathbb{R}^M$. In fact, this problem could be cast as an $M - 1$ -dimensional unconstrained optimization over $\tilde{\mathbf{w}}_{s_t} \in \mathbb{R}^{M-1}$ such that the M th component of the weight vector complements the sum of the entire vector to be 1, i.e.,

$$\begin{aligned}\min_{\tilde{\mathbf{w}}_{s_t} \in \mathbb{R}^{M-1}} \quad & \ell \left(y_t, \sum_{i=1}^{M-1} w_{s_t^E}^{(i)} \hat{y}_t^{(i)} \right) \\ \text{subject to} \quad & w_{s_t^E}^{(M)} = 1 - \tilde{\mathbf{w}}_{s_t}^T \mathbf{1}.\end{aligned} \quad (6)$$

With the transformation of the constrained problem from (5), which is a linearly constrained quadratic optimization problem under the squared loss, i.e., $\ell(y_t, \hat{y}_t) = \mathbb{E}[(y_t - \hat{y}_t)^2 | s_t^E]$, to (6), which is an unconstrained problem, the optimal affine ensembling weight vector admits a closed-form solution given as

$$\mathbf{w}_{s_t^E, \text{aff}}^* = \mathbf{C}(s_t)^{-1} \mathbf{a}(s_t) - \frac{\mathbf{1}^T \mathbf{C}(s_t)^{-1} \mathbf{a}(s_t) - 1}{\mathbf{1}^T \mathbf{C}(s_t)^{-1} \mathbf{1}} \mathbf{C}(s_t)^{-1} \mathbf{1}.$$

We note that the affine combination preserves the unbiasedness of the base predictors. If each base predictor provides unbiased predictions of y_t , then we have

$$\begin{aligned}\mathbb{E}[\mathbf{w}_{s_t^E}^T \hat{\mathbf{y}}_t | s_t^E] &= \sum_{i=1}^M w_{s_t^E}^{(i)} \mathbb{E}[\hat{y}_t^{(i)}] = \sum_{i=1}^M w_{s_t^E}^{(i)} \mathbb{E}[y_t] \\ &= \mathbb{E}[y_t] \sum_{i=1}^M w_{s_t^E}^{(i)} = \mathbb{E}[y_t].\end{aligned}$$

Hence, for the expected squared error loss, the optimal affine-constrained weights are given as

$$\begin{aligned}L_{t, \text{aff}}^* &:= \sigma(s_t)^2 - \mathbf{a}(s_t)^T \mathbf{C}(s_t)^{-1} \mathbf{a}(s_t) \\ &\quad + \frac{(\mathbf{1}^T \mathbf{C}(s_t)^{-1} \mathbf{a}(s_t) - 1)^2}{\mathbf{1}^T \mathbf{C}(s_t)^{-1} \mathbf{1}}.\end{aligned}$$

As for the convex constraint case, we not only require that the components of the weight vector sum up to 1 but also they should be nonnegative, i.e., the optimization problem becomes

$$\begin{aligned}\min_{\mathbf{w}_{s_t^E} \in \mathbb{R}^M} \quad & \ell \left(y_t, \sum_{i=1}^M w_{s_t^E}^{(i)} \hat{y}_t^{(i)} \right) \\ \text{subject to} \quad & \mathbf{w}_{s_t^E}^T \mathbf{1} = 1 \ \& \ w_{s_t^E}^{(i)} \geq 0, \quad i = 1, \dots, M.\end{aligned} \quad (7)$$

Under a convex loss ℓ , problem (7) is a convex quadratic minimization problem, and the feasible region is the M -dimensional unit simplex. Unlike the previous two constraints, (7) does not admit a closed-form solution. We can, however, project (unconstrained) weights to the unit simplex iteratively to find the optimal weight vector. For brevity, we let $M = 2$, which is a

common case in practice, and derive the procedure under squared error loss. To this end, we begin by noting that the squared loss at time t as a function of the ensembling weight vector $\mathbf{w}_{s_t^E}$ is

$$\begin{aligned} L_t(\mathbf{w}_{s_t^E}) &= \sigma(s_t)^2 - \mathbf{a}(s_t)^T \mathbf{C}(s_t)^{-1} \mathbf{a}(s_t) \\ &\quad + (\mathbf{w}_{s_t^E} - \mathbf{C}(s_t)^{-1} \mathbf{a}(s_t))^T \mathbf{C}(s_t) (\mathbf{w}_{s_t^E} \\ &\quad - \mathbf{C}(s_t)^{-1} \mathbf{a}(s_t)) \\ &= L_{t,unc}^* + (\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,unc}^*)^T \mathbf{C}(s_t) \\ &\quad \times (\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,unc}^*). \end{aligned} \quad (8)$$

We can rewrite (8) instead of the optimal affine weights $\mathbf{w}_{s_t^E,aff}^*$ as

$$\begin{aligned} L_t(\mathbf{w}_{s_t^E}) &= L_{t,unc}^* + \left((\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*) \right. \\ &\quad \left. + (\mathbf{w}_{s_t^E,aff}^* - \mathbf{w}_{s_t^E,unc}^*) \right)^T \mathbf{C} \\ &\quad \times (s_t) \left((\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*) + (\mathbf{w}_{s_t^E,aff}^* - \mathbf{w}_{s_t^E,unc}^*) \right) \\ &= L_{t,unc}^* + (\mathbf{w}_{s_t^E,aff}^* - \mathbf{w}_{s_t^E,unc}^*)^T \mathbf{C}(s_t) \\ &\quad \times (\mathbf{w}_{s_t^E,aff}^* - \mathbf{w}_{s_t^E,unc}^*) + (\mathbf{w}_{s_t^E} \\ &\quad - \mathbf{w}_{s_t^E,aff}^*)^T \mathbf{C}(s_t) (\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*) \\ &\quad - 2 (\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*)^T \mathbf{C}(s_t) (\mathbf{w}_{s_t^E,aff}^* - \mathbf{w}_{s_t^E,unc}^*) \\ &= L_{t,aff}^* + (\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*)^T \mathbf{C}(s_t) (\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*) \\ &\quad - 2 \frac{(\mathbf{1}^T \mathbf{w}_{s_t^E,unc}^* - 1)}{\mathbf{1}^T \mathbf{C}(s_t)^{-1} \mathbf{1}} (\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*)^T \mathbf{1} \\ &= L_{t,aff}^* + (\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*)^T \mathbf{C}(s_t) (\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*) \end{aligned}$$

where the last line follows from $(\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*)^T \mathbf{1} = 0$ due to the unit simplex constraint. Further ignoring the constant $L_{t,aff}^*$, the optimization problem reduces to minimizing the $\mathbf{C}(s_t)$ -weighted ℓ_2 -norm of $\mathbf{w}_{s_t^E} - \mathbf{w}_{s_t^E,aff}^*$ subject to $\mathbf{w}_{s_t^E}$ being on the unit simplex. For the assumed $M = 2$, we let Δ_2 be the unit simplex in $\mathbb{R}^2$. There are two cases to consider for this problem.

- $\mathbf{w}_{s_t^E,aff}^* \in \Delta_2$: The optimal affine combination is within the unit simplex set, and hence, a closed-form solution can be written for the optimal weight vector: $\mathbf{w}_{s_t^E,con}^* = \mathbf{w}_{s_t^E,aff}^*$. Accordingly, $L_{t,con}^* := L_{t,aff}^*$.
- $\mathbf{w}_{s_t^E,aff}^* \notin \Delta_2$: This case is when combination weights are of opposite signs. For simplicity, we assume the weight that is assigned to the first base predictor is negative and the other one is positive. Then, we can write $\mathbf{w}_{s_t^E,con}^* = \mathbf{w}_{s_t^E,aff}^* + \alpha[1 \ -1]^T$ for some $\alpha \in \mathbb{R}$. Consequently, the cost function to minimize becomes the $\mathbf{C}(s_t)$ -weighted ℓ_2 -norm of $\alpha[-1 \ 1]^T$. This implies that the cost is proportional to the magnitude of α . The optimal choice

would be the smallest value that keeps $\mathbf{w}_{s_t^E,con}^*$ within Δ_2 , which is $-w_{s_t^E,aff}^{(1)}$. In this case, $\mathbf{w}_{s_t^E,con}^*$ is the corner point of Δ_2 : $[0 \ 1]^T$. This leads to $L_{t,con}^* := L_{t,aff}^* + [-1 \ 1] \mathbf{C}(s_t) [-1 \ 1]^T (w_{s_t^E,aff}^{(1)})^2$.

Remark From the inclusion order of the feasible regions of the constrained problems as well as the derived optimal squared error losses as the special cases, we naturally have

$$L_{t,unc}^* \leq L_{t,aff}^* \leq L_{t,con}^* \quad (9)$$

for all times t . This inequality holds in theory when the correlation statistics of the target signal and the base models are known at all times. In practice, however, those statistics are rarely known and, therefore, there is a learning cost associated with each constraint, which renders the relation in (9) useless. Hence, even though the unconstrained case seems to promise the least error, the associated learning procedure might lead to overfitting, i.e., in essence, affine and convex constrained cases perform as “regulators” in that they offer a tradeoff between bias and variance of the ensembling model. In fact, as shown in our simulations in Section V, the unconstrained ensembling scheme does not always achieve the lowest error.

Remark: Since $\mathbf{C}(s_t)$ and $\mathbf{a}(s_t)$ are rarely known in practice, we present novel and generic ensemble algorithms to find the optimal ensembling weight vectors under three constraint schemes, in Section IV.

APPENDIX B ENSEMBLE LEARNING PROCEDURE

Here, we provide a detailed view of our ensemble learning procedure as illustrated in Fig. 2.

APPENDIX C ENSEMBLE DERIVATIONS

A. LightGBM Ensemble Derivations

1) *Unconstrained Case:* For the unconstrained case, there is no restriction on the value of $w_{s_t^E}^{(i)}$. Therefore, we can simply write the transformation relation as $w_{s_t^E}^{(i)} = p_{s_t^E}^{(i)}$. The gradient and Hessian equations in (3) for the base learner i , at time t become

$$\begin{aligned} G_i &= 2(y_t - \hat{y}_t^E) \left(-\frac{\partial \hat{y}_t^E}{\partial p_{s_t^E}^{(i)}} \right) \\ &= 2(y_t - \hat{y}_t^E) \left(-\sum_{m=1}^M \frac{\partial w_{s_t^E}^{(m)}}{\partial p_{s_t^E}^{(i)}} \hat{y}_t^{(m)} \right) \\ &= 2\hat{y}_t^{(i)} (\hat{y}_t^E - y_t) \\ H_i &= 2\hat{y}_t^{(i)} \left(\frac{\partial \hat{y}_t^E}{\partial p_{s_t^E}^{(i)}} \right) \\ &= 2(\hat{y}_t^{(i)})^2. \end{aligned}$$

Algorithm 1: Ensemble Learning Procedure.**• Offline Phase:** Training the Base and Ensemble Algorithms**inputs:**

- $\{y_t\} \triangleq \{y_t\}_{t=1}^T$: the training data that consists of T sequential samples from the time series sequence y_t .
- M base models.
- $\{s_t^{(i)}\} \triangleq \{s_t^{(i)}\}_{t=1}^T$: time series features for the i th base predictor at time T .

outputs:

- ensemble learner, which produces the weight vector $w_{s_t^E}$ for combining base predictions.
- $\{s_t^E\} \triangleq \{s_t^E\}_{t=1}^T$: time series features for the ensemble learner.

prepare ensemble data :

- split $\{y_t\} \triangleq \{y_t\}_{t=1}^T$ into training and test periods for the base algorithms, where $1 \leq t \leq T_1$ is the training period and $T_1 + 1 \leq t \leq T$ is the test period.
- train all of the base algorithms over the training period $1 \leq t \leq T_1$.
- generate forecasts for the test period for all of the base models, i.e., form $\{\hat{y}_t^{(i)}\}_{t=T_1+1}^T$, $1 \leq i \leq M$.
- construct the ensemble side-information vector s_t^E , which is usually the union of the base models' side-information vectors.

train the ensemble learner :

- train the ensemble learner using s_t^E to minimize the loss

$$\arg \min_{w_{s_t^E}} \sum_{t=T_1+1}^T \ell(y_t, \hat{y}_t^E) \quad (10)$$

where $\hat{y}_t^E = \sum_{i=1}^M w_{s_t^E}^{(i)} \hat{y}_t^{(i)}$, ℓ is a given differentiable loss function such as the squared error loss,

$w_{s_t^E} = [w_{s_t^E}^{(1)}, w_{s_t^E}^{(2)}, \dots, w_{s_t^E}^{(M)}]^T$ is the weight vector for the linear combination of the base learners, and $w_{s_t^E}^{(i)}$ is the combination weight assigned to the i th base learner at time t .

- train all of the base algorithms over the entire training data $1 \leq t \leq T$.

• Online Phase: Predicting the Test Data**inputs:**

- $\{y_t\}_{t=T+1}^{T_2}$: the test data that consists of sequential data samples of the time series sequence y_t , from $T + 1$ to T_2 .
- the trained ensemble learner from the offline phase.
- M base models, trained on the training data.
- $\{s_t^{(i)}\}_{t=T+1}^{T_2}$: time series features for the i th base predictor at time T .
- $\{s_t^E\}_{t=T+1}^{T_2}$: time series features for the ensemble learner.

outputs:

- combination weights $w_{s_t^E}^{(i)}$ for each base learner $1 \leq i \leq M$, for $T + 1 \leq t \leq T_2$.

- ensemble predictions $\hat{y}_t^E$ for $T + 1 \leq t \leq T_2$.

generate combination weights:

- generate forecasts for the test data from all of the base models, i.e., generate $\{\hat{y}_t^{(i)}\}_{t=T+1}^{T_2}$, $1 \leq i \leq M$.
- construct the ensemble side-information vector s_t^E for the test dataset.
- feed the side-information vector s_t^E to the ensemble learner to generate the weight vector $w_{s_t^E}^T = [w_{s_t^E}^{(1)}, w_{s_t^E}^{(2)}, \dots, w_{s_t^E}^{(M)}]^T$, for $T + 1 \leq t \leq T_2$.
- linearly combine the weights given to each base learner with corresponding predictions to predict $\hat{y}_t^E = \sum_{i=1}^M w_{s_t^E}^{(i)} \hat{y}_t^{(i)}$, for $T + 1 \leq t \leq T_2$.

2) *Affine Constrained Case:* For the affine constrained case, the restriction is the summation of the combination weights should be equal to one, i.e., $\sum_{i=1}^M w_{s_t^E}^{(i)} = 1$. In order to satisfy this relation, we apply the given normalization operation $w_{s_t^E}^{(i)} = \frac{p_{s_t^E}^{(i)}}{\sum_{m=1}^M p_{s_t^E}^{(m)}}$. Hence, the gradient and Hessian equations in (3) for the base learner i , at time t become

$$\begin{aligned} G_i &= 2(y_t - \hat{y}_t^E) \left(-\frac{\partial \hat{y}_t^E}{\partial p_{s_t^E}^{(i)}} \right) \\ &= 2(y_t - \hat{y}_t^E) \left(-\sum_{m=1}^M \frac{\partial w_{s_t^E}^{(m)}}{\partial p_{s_t^E}^{(i)}} \hat{y}_t^{(m)} \right) \\ &= 2(y_t - \hat{y}_t^E) \left(\frac{1}{\sum_{m=1}^M p_{s_t^E}^{(m)}} \right) (\hat{y}_t^E - \hat{y}_t^{(i)}) \\ H_i &= 2 \left(\frac{1}{\sum_{m=1}^M p_{s_t^E}^{(m)}} \right)^2 (\hat{y}_t^E - \hat{y}_t^{(i)}) (3\hat{y}_t^E - 2y_t - \hat{y}_t^{(i)}). \end{aligned}$$

3) *Convex Constrained Case:* For the convex constrained case, the restriction is the summation of the combination weights should be equal to one, such that $\sum_{i=1}^M w_{s_t^E}^{(i)} = 1$. In addition, $0 \leq w_{s_t^E}^{(i)} \leq 1$ should also be satisfied. Therefore, we apply the softmax transformation given as $w_{s_t^E}^{(i)} = \frac{e^{p_{s_t^E}^{(i)}}}{\sum_{m=1}^M e^{p_{s_t^E}^{(m)}}}$. Therefore, the gradient and Hessian equations in (3) for the base learner i , at time t become

$$\begin{aligned} G_i &= 2(y_t - \hat{y}_t^E) \left(-\frac{\partial \hat{y}_t^E}{\partial p_{s_t^E}^{(i)}} \right) \\ &= 2(y_t - \hat{y}_t^E) \left(-\sum_{m=1}^M \frac{\partial w_{s_t^E}^{(m)}}{\partial p_{s_t^E}^{(i)}} \hat{y}_t^{(m)} \right) \end{aligned}$$

$$= 2 (y_t - \hat{y}_t^E) w_{s_t^E}^{(i)} (\hat{y}_t^E - \hat{y}_t^{(i)})$$

$$H_i = G_i \left(1 - 2w_{s_t^E}^{(i)} \right) + 2 \left(w_{s_t^E}^{(i)} \right)^2 \left(\hat{y}_t^E - \hat{y}_t^{(i)} \right)^2.$$

B. MLP Ensemble Derivations

1) *Unconstrained Case:* For the unconstrained case, there is no restriction on the value of $w_{s_t^E}^{(i)}$. Therefore, we can simply write the transformation relation as $w_{s_t^E}^{(i)} = p_{s_t^E}^{(i)}$. Hence, the backpropagation equations become

$$\frac{\partial L_t}{\partial U_{l,k}^{(1)}} = \sum_{i=1}^M 2(\hat{y}_t^E - y_t) \hat{y}_t^{(i)} U_{m,l}^{(2)} f'(v_l) x_k$$

$$\frac{\partial L_t}{\partial U_{m,l}^{(2)}} = 2(\hat{y}_t^E - y_t) \hat{y}_t^{(m)} z_l$$

where $f'(v_l)$ is the derivative of ReLU, which is the piecewise function

$$f'(v_l) = \begin{cases} 0, & \text{if } v_l \leq 0 \\ v_l, & \text{if } v_l > 0. \end{cases}$$

2) *Affine Constrained Case:* For the affine constrained case, the restriction is the summation of the combination weights should be equal to one, such that $\sum_{i=1}^M w_{s_t^E}^{(i)} = 1$. In order to satisfy this relation, we apply the given normalization operation $w_{s_t^E}^{(i)} = \frac{p_{s_t^E}^{(i)}}{\sum_{m=1}^M p_{s_t^E}^{(m)}}$. Hence, the backpropagation equations become

$$\frac{\partial L_t}{\partial U_{l,k}^{(1)}} = \sum_{i=1}^M 2c (\hat{y}_t^E - y_t) (\hat{y}_t^{(m)} - \hat{y}_t^E) U_{m,l}^{(2)} f'(v_l) x_k$$

$$\frac{\partial L_t}{\partial U_{m,l}^{(2)}} = 2c (\hat{y}_t^E - y_t) (\hat{y}_t^{(m)} - \hat{y}_t^E) z_l$$

where c is the constant term given as $c = \left(\frac{1}{\sum_{m=1}^M p_{s_t^E}^{(m)}} \right)$.

3) *Convex Constrained Case:* In the convex constrained case, the restriction is the summation of the combination weights should be equal to one, such that $\sum_{i=1}^M w_{s_t^E}^{(i)} = 1$. In addition, $0 \leq w_{s_t^E}^{(i)} \leq 1$ should also be satisfied. Therefore, we apply the given softmax transformation $w_{s_t^E}^{(i)} = \frac{e^{\frac{p_{s_t^E}^{(i)}}{p_{s_t^E}^{(m)}}}}{\sum_{m=1}^M e^{\frac{p_{s_t^E}^{(i)}}{p_{s_t^E}^{(m)}}}}$. Hence, the backpropagation equations become

$$\frac{\partial L_t}{\partial U_{l,k}^{(1)}} = \sum_{i=1}^M 2 (\hat{y}_t^E - y_t) w_m (\hat{y}_t^{(m)} - \hat{y}_t^E) U_{m,l}^{(2)} f'(v_l) x_k$$

$$\frac{\partial L_t}{\partial U_{m,l}^{(2)}} = 2 (\hat{y}_t^E - y_t) w_m (\hat{y}_t^{(m)} - \hat{y}_t^E) z_l.$$

REFERENCES

- [1] X. Wang et al., "Learning nonstationary time-series with dynamic pattern extractions," *IEEE Trans. Artif. Intell.*, vol. 3, no. 5, pp. 778–787, Oct. 2022.
- [2] R. Lou, Z. Lv, and M. Guizani, "Wave height prediction suitable for maritime transportation based on green ocean of things," *IEEE Trans. Artif. Intell.*, vol. 4, no. 2, pp. 328–337, Apr. 2023.
- [3] W. Liao, Z. Yang, X. Chen, and Y. Li, "WindGMMN: Scenario forecasting for wind power using generative moment matching networks," *IEEE Trans. Artif. Intell.*, vol. 3, no. 5, pp. 843–850, Oct. 2022.
- [4] R. T. Clemen, "Combining forecasts: A review and annotated bibliography," *Int. J. Forecasting*, vol. 5, no. 4, pp. 559–583, 1989.
- [5] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems*. Berlin, Germany: Springer, 2000, pp. 1–15.
- [6] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 competition: 100,000 time series and 61 forecasting methods," *Int. J. Forecasting*, vol. 36, no. 1, pp. 54–74, 2020.
- [7] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *Int. J. Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.
- [8] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [9] L. Breiman, "Bagging predictors," *Mach. Learn.*, vol. 24, no. 2, pp. 123–140, 1996.
- [10] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [11] P. Montero-Manso, G. Athanasopoulos, R. J. Hyndman, and T. S. Talagala, "FFORMA: Feature-based forecast model averaging," *Int. J. Forecasting*, vol. 36, no. 1, pp. 86–92, 2020.
- [12] S. M. Mastelini, F. K. Nakano, C. Vens, and A. C. P. d. L. F. d. Carvalho, "Online extra trees regressor," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Oct. 21, 2022, doi: [10.1109/TNNLS.2022.3212859](https://doi.org/10.1109/TNNLS.2022.3212859).
- [13] R. M. Cruz, R. Sabourin, G. D. Cavalcanti, and T. I. Ren, "META-DES: A dynamic ensemble selection framework using meta-learning," *Pattern Recognit.*, vol. 48, no. 5, pp. 1925–1935, 2015.
- [14] J. Smith and K. F. Wallis, "A simple explanation of the forecast combination puzzle," *Oxford Bull. Econ. Statist.*, vol. 71, no. 3, pp. 331–355, 2009.
- [15] Z.-H. Zhou, J. Wu, and W. Tang, "Ensembling neural networks: Many could be better than all," *Artif. Intell.*, vol. 137, no. 1/2, pp. 239–263, 2002.
- [16] X. Qiu, L. Zhang, Y. Ren, P. N. Suganthan, and G. Amarantunga, "Ensemble deep learning for regression and time series forecasting," in *Proc. IEEE Symp. Comput. Intell. Ensemble Learn.*, 2014, pp. 1–6.
- [17] J. Mendes-Moreira, C. Soares, A. M. Jorge, and J. F. D. Sousa, "Ensemble approaches for regression: A survey," *ACM Comput. Surv.*, vol. 45, no. 1, pp. 1–40, 2012.
- [18] H. Tyralis, G. Papacharalampous, and A. Langousis, "Super ensemble learning for daily streamflow forecasting: Large-scale demonstration and comparison with multiple machine learning algorithms," *Neural Comput. Appl.*, vol. 33, no. 8, pp. 3053–3068, 2021.
- [19] F. Ilhan, O. Karaahmetoglu, I. Balaban, and S. S. Kozat, "Markovian RNN: An adaptive time series prediction network with HMM-based switching for nonstationary environments," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 2, pp. 715–728, Feb. 2023.
- [20] T. Moon and W. Stirling, *Mathematical Methods and Algorithms for Signal Processing*. Englewood Cliffs, NJ, USA: Prentice Hall, 2000.
- [21] P. Jain and P. Kar, "Non-convex optimization for machine learning," *Foundations Trends Mach. Learn.*, vol. 10, no. 3/4, pp. 142–363, 2017.
- [22] S. Tao, "Deep neural network ensembles," in *Proc. Int. Conf. Mach. Learn., Optim., Data Sci.*, 2019, pp. 1–12.
- [23] T. Hospedales, A. Antoniou, P. Micalelli, and A. Storkey, "Meta-learning in neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 9, pp. 5149–5169, Sep. 2022.
- [24] S. Ma and R. Fildes, "Retail sales forecasting with meta-learning," *Eur. J. Oper. Res.*, vol. 288, no. 1, pp. 111–128, 2021.
- [25] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. 31st Annu. Conf. Neural Inf. Process. Syst.*, 2017, pp. 3149–3157.
- [26] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.

- [27] J. Lee, W. Wang, F. Harrou, and Y. Sun, "Wind power prediction using ensemble learning-based models," *IEEE Access*, vol. 8, pp. 61517–61527, 2020.
- [28] S. T. U. Raju et al., "An approach for demand forecasting in steel industries using ensemble learning," *Complexity*, vol. 2022, pp. 1–19, 2022.
- [29] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 785–794.
- [30] O. Akbilgic, H. Bozdogan, and M. E. Balaban, "A novel hybrid RBF neural networks model as a forecaster," *Statist. Comput.*, vol. 24, pp. 365–375, 2014.
- [31] P. Derbeko, R. El-Yaniv, and R. Meir, "Variance optimized bagging," in *Proc. 13th Eur. Conf. Mach. Learn.*, 2002, pp. 60–72.
- [32] S. Mao, L. Jiao, L. Xiong, S. Gou, B. Chen, and S.-K. Yeung, "Weighted classifier ensemble based on quadratic form," *Pattern Recognit.*, vol. 48, no. 5, pp. 1688–1706, 2015.
- [33] J. Yang, X. Zeng, S. Zhong, and S. Wu, "Effective neural network ensemble approach for improving generalization performance," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 24, no. 6, pp. 878–887, Jun. 2013.
- [34] A. K. Jain, J. Mao, and K. M. Mohiuddin, "Artificial neural networks: A tutorial," *Computer*, vol. 29, no. 3, pp. 31–44, 1996.
- [35] R. Lin, Z. Zhou, S. You, R. Rao, and C.-C. J. Kuo, "Geometrical interpretation and design of multilayer perceptrons," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Jul. 21, 2022, doi: [10.1109/TNNLS.2022.3190364](https://doi.org/10.1109/TNNLS.2022.3190364).
- [36] G. A. Seber and A. J. Lee, *Linear Regression Analysis*. Hoboken, NJ, USA: Wiley, 2012.
- [37] D. Dua and C. Graff, "UCI machine learning repository," 2021. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [38] G. E. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*. Hoboken, NJ, USA: Wiley, 2015.
- [39] W. Zaremba, I. Sutskever, and O. Vinyals, "Recurrent neural network regularization," 2014, *arXiv:1409.2329*.
- [40] K. Greff, R.K. Srivastava, J. Koutnfk, B. R. Steunebrink, and J. Schmidhuber, "LSTM: A search space odyssey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 10, pp. 2222–2232, Oct. 2017.
- [41] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
- [42] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*. Melbourne, Australia: OTexts, 2018.